

OOP-4. SOLID

4. SOLID

객체지향에서 좋은 설계와 아키텍처를 이야기하면 빠지지 않고 나오는 개념이 있다. 바로 **SOLID**이다.

SOLID 5원칙

- 단일 책임 원칙 SRP (Single Responsibility Principle)
 - 개방 폐쇄 원칙 OCP (Open - Closed Principle)
 - 리스코프 치환 원칙 LSP (Liskov Substitution Principle)
 - 인터페이스 분리 원칙 ISP (Interface Segregation Principle)
 - 의존성 역전 원칙 DIP (Dependency Inversion Principle)
-

각 원칙은 객체지향 언어에서 좋은 설계를 얻기 위해 개발자가 지켜야 할 규범과 같은 것을 이야기한다.
각 원칙의 **목표**는 소프트웨어의 **유지보수성**과 **확장성**을 높이는 것이다.

설계 관점에서 코드의 유지보수성을 판단할 때 사용할 수 있는 조금 더 실무적인 세 가지 맥락이 있다.

-  **영향 범위**: 코드 변경으로 인한 영향 범위가 어떻게 되는가?
-  **의존성**: 소프트웨어의 의존성 관리가 제대로 이뤄지고 있는가?
-  **확장성**: 쉽게 확장 가능한가?

SOLID를 따르는 코드는 **코드 변경으로 인한 영향 범위를 축소**할 수 있고, 의존성을 제대로 관리하며 기능 **확장**이 쉽다.

그래서 코드의 **유지보수성**이 올라간다.

4.1 SOLID 소개

4.1.1 단일 책임 원칙 : SRP (Single Responsibility Principle)

단일 책임 원칙은 클래스에 너무 많은 책임이 할당돼서는 안 되며, 단 하나의 책임만 가져야 한다는 원칙이다.

긴 코드와 책임의 관계

- 코드의 양이 곧 책임의 크기를 의미하지는 않는다.

- 하지만 일반적으로 코드가 길어질수록 책임 분리가 제대로 되지 않은 경우가 많다.
 - 긴 코드는 가독성을 해치고, 메소드가 어디까지 영향을 주는지 파악하기 어렵다.
-

책임이 과한 클래스의 문제

- 복잡하고 많은 책임을 가진 클래스는 **변경 시 문제**를 일으킨다.
 - 영향 범위를 알 수 없기 때문에 코드 변경이 위험하고 어렵다.
-

단일 책임 원칙의 의미

- 클래스는 **특정 역할 달성에만 집중**해야 한다.
 - 책임이 하나라면:
 - 코드를 이해하기 쉽다.
 - 수정 시 특정 클래스/모듈만 고치면 된다.
 - 다른 책임과의 충돌을 걱정할 필요가 없다.
-

단일 책임 원칙의 목적

- **변경으로 인한 영향 범위를 최소화**한다.
 - 소프트웨어는 복잡계이므로 요구사항 변경이 빈번하다.
 - 그럼에도 **외부 변경 요청에도 시스템의 항상성을 유지**하는 것이 SRP의 가장 큰 목적이다.
-

그렇다면 여기서 말하는 “책임”이란 무엇일까?

```
class Developer {  
    public String createFrontendCode() { /* 프론트엔드 코드를 만든다 */ }  
    public String publishFrontend() { /* 프론트엔드 서비스 배포한다 */ }  
    public String createBackendCode() { /* 백엔드 코드를 만든다 */ }  
    public String serveBackend() { /* 백엔드 서비스 배포한다 */ }  
}
```

이는 개발자를 표현하는 클래스이다.

이 코드는 단일 책임 원칙을 위배하고 있는가에 대해서 잠시 생각해보자.

1) 단일 책임 원칙을 위배한다고 보는 입장

왜냐하면 개발자 시선에서 봤을 때 개발자는 프론트와 백으로 분류되고, 그렇기 때문에 두 역할을 모두 하고 있는

코드에는 두 개의 책임이 있다고 보기 때문이다.

그렇기 때문에 책임을 모두 갖고 있는 위의 코드는 위배된다고 보는 입장이다.

2) 책임을 프론트엔드 개발자, 백엔드 개발자, 시스템 운영자로 분류

1. 프론트엔드 개발자의 책임 : `createFrontendCode`
2. 백엔드 개발자의 책임 : `createBackendCode`
3. 시스템 운영자의 책임 : `publishFrontend` , `serveBackend`

3) 개발자를 구분하지 않고 하나의 시스템 개발하는 사람으로 본다면?

- 시스템 개발자의 책임 : `createFrontendCode` , `createBackendCode` , `publishFrontend` , `serveBackend`

이러한 이유로 단일 책임 원칙은 굉장히 단순한 원칙처럼 보이지만, 협업하는 실무레벨에서 이 원칙을 적용하는 것은

꽤나 어려운 일이다.

왜냐하면 책임은 문맥을 포함하는 개념이기 때문이며 그로 인해 책임이라는 개념은 그것을 바라보는 개인이나 상황마다

다르게 해석될 여지가 있기 때문이다.

따라서 책임이란 무엇이고 이를 어떻게 나눌지 기준이 필요하다.

이 같은 배경에서 SOLID의 창시자인 로버트 C. 마틴은 다음과 같이 첨언한다.

💡 "하나의 모듈은 하나의, 오직 하나의 액터에 대해서만 책임져야 한다"

책임을 설명하기 위해 **액터**라는 존재가 등장했다.

액터는 메시지를 전달하는 주체이고 단일 책임 원칙에서 말하는 책임은 액터에 대한 책임이다.

즉, 단일 책임 원칙을 이해하려면 오히려 액터에 집중해야 한다.

메시지를 요청하는 주체가 누구냐에 따라 책임이 달라질 수 있기 때문이다.

어떤 클래스를 사용하게 될 액터가 한 명이라면 단일 책임 원칙을 지키고 있는 것이고 여럿이라면 위반하고 있는 것이다.

따라서 단일 책임 원칙을 이해하려면 시스템에 존재하는 액터를 먼저 이해해야 한다.

또한 단일 책임 원칙에서 말하는 책임은 결국 액터에 대한 책임이기 때문이다.

🎯 단일 책임 원칙의 목표

- 클래스가 변경됐을 때 영향을 받는 액터가 하나여야 한다.
- 클래스를 변경할 이유는 유일한 액터의 요구사항이 변경될 때로 제한되어야 한다.

4.1.2 개방 폐쇄 원칙 OCP (Open-Closed Principle)

"확장에는 열려있고(open) 변경에는 닫혀있어야(closed) 한다" 라는 말로 표현되기도 한다.

이 원칙의 주된 목적은 기존 코드를 수정하지 않으면서도 확장이 가능한 시스템을 만드는 것이다.

그렇다면 기존 코드를 수정하지 않으면서도 확장이 가능한 시스템을 만들어야 하는 이유는 무엇일까? 시스템을 운영하면서 코드를 변경하는 것은 매우 위험한 일이기 때문이다.

작은 프로젝트는 코드 수정이 자유롭지만, 규모가 큰 시스템에서는 코드를 변경하는 것이 쉬운 일이 아니다. 특히나 변경하려는 코드를 사용하려는 액터가 여럿이면 더더욱 그렇다.

따라서 코드를 확장하고자 할 때 취할 수 있는 최고의 전략은 기존 코드를 아예 건드리지 않는 것이다. 근데 이게 무슨 말일까? 이게 가능하긴 할까?

1) 주문 클래스가 음식이라는 구현체를 직접 사용했던 코드

```
Error parsing Mermaid diagram!

Cannot read properties of null (reading 'getBoundingClientRect')
```

새로운 요구사항이 들어오면 Order 클래스를 변경해야 했다. 즉, 구현에 의존하는 코드를 만들었더니 새로운 요구사항이 생기자 새로운 구현을 지원하는 코드로 변경해야 했던 것이다.

2) 주문 클래스가 계산 가능한이라는 역할을 사용했던 코드

```
Order -----> Calculable <|----- Food
                  <|----- BrandProduct : 새로운 요구사항
```

역할에 의존했더니 새로운 요구사항이 들어와도 Order 클래스가 영향을 받지 않는다.

OCP의 목표는 확장하기 쉬우면서도 변경으로 인한 영향범위를 최소화하는 것이다. 또한 코드를 추상화된 역할에 의존하게 만듦으로써 이를 달성할 수 있다.

4.1.3 리스코프 치환 원칙 : 부모 역할을 하던 걸 자식이 대신해도 아무 문제 없어야 한다.

이 원칙은 한 문장으로 정의하면 '기본 클래스의 계약을 파생 클래스가 제대로 치환할 수 있는지 확인하라'는 원칙이다.

예시:

```
class Rectangle {
    protected long width;
    protected long height;

    public long calc(){
        return width * height;
    }
}

class Square extends Rectangle {
```

```

    public Square(long length){
        super(length, length);
    }
}

```

```

Rectangle rectangle = new Square(10);
rectangle.setHeight(5);

System.out.println(rectangle.calc()); // 코드의 실행 결과는 5

```

기본 클래스 Rectangle로 만들어진 rectangle 변수에 파생 클래스인 Square의 인스턴스를 할당했다.
10×10 크기의 정사각형을 만들어 넣어줬다.
그리고 곧바로 높이를 5로 변경하였다.

하지만 이럴 때 정사각형은 어떻게 동작해야 할까?
5×5로 바뀌어야 할까 아니면 더 이상 정사각형으로 취급하지 않아야 할까?

이 예시는 리스코프 치환원칙의 위반사례이다.

파생 클래스가 기본 클래스의 모든 동작을 완전히 대체할 수 있어야 한다.

하지만 현재 정사각형 클래스는 직사각형 클래스의 모든 동작을 완전히 대체하지 못한다.

우선, 파생 클래스가 기본 클래스를 대체할 수 있는지 파악하기 위해 기본 클래스에 할당된 의도가 무엇인지를 먼저 파악해야 한다.

Rectangle 클래스에 할당된 의도

- getWidth를 호출하면 너비 값을 반환한다
- getHeight를 호출하면 높이 값을 반환한다
- setWidth를 호출하면 너비 값을 변경한다
- setHeight를 호출하면 높이 값을 변경한다
- calc를 호출하면 넓이 값을 계산한다.

파생 클래스는 기본 클래스에서 정의한 의도를 모두 지킬 수 있어야 한다.

예시) 파생 클래스가 기본 클래스에서 정의한 의도를 지키도록 메소드를 오버라이딩

```

class Square extends Rectangle {
    public Square(long length) {
        super(length, length);
    }

    @Override
    public void setWidth(long width) {
        super.width = width;
        super.height = width;
    }

    @Override
    public void setHeight(long height){
        super.width = height;
    }
}

```

```
        super.height = height;
    }
}
```

이제 이 코드는 앞에서 도형의 높이를 변경하고 넓이를 계산하는 코드에서도 그럴싸하게 동작할 것이다. 즉 높이를 변경하면 모든 값이 동일하게 변경되면서 정사각형으로 변경될 것이다. 그렇다면 이제 Square는 기본 클래스를 대체할 수 있는 것일까?

일반적으로 세터는 다른 속성은 건드리지 않고 내가 원하는 특정 속성 값만 그대로 치환하는 역할을 하므로 이 코드마저 기본 클래스의 의도를 위반한 것일 수 있기 때문이다.

어떤 개발자가 rectangle이라는 객체를 매개변수로 받아 이 객체의 setHeight 메소드를 실행해 도형의 높이 값을 변경하는데
너비까지 변경되는 뜬금없는 상황이 발생한다.

전후 사정을 알고 있는 개발자는 내부 흐름을 알지만, 맥락을 모르는 개발자가 코드를 보면 이상하다고 느낄 수 있다.

그렇다면 초기 코드 작성자의 의도를 파악하려면 어떻게 해야 할까?
원시적으로는 코드 작성자에게 직접 물어보는 방법이 있지만 한계가 발생한다.

대신 코드 작성자의 의도를 드러낼 수 있는 방법이 **테스트 코드**를 사용하는 것이다.
초기 코드 작성자가 생각하는 모든 의도를 테스트 코드로 만들어 두는 것이다.
이렇게 할 수 있다면 파생 클래스를 작성하는 개발자는 테스트를 보고 초기 코드 작성자의 의도를 파악할 수 있을 것이고, 기본 클래스로 작성된 테스트에
파생 클래스를 추가해 테스트해볼 수 있을 것이다.

인터페이스는 계약이며, 테스트는 계약 명세이다.

4.1.4 인터페이스 분리원칙

알겠습니다. 말씀해주신 내용을 흐름은 유지하면서 더 매끄럽게 다듬어드릴게요.

인터페이스 분리 원칙 (ISP, Interface Segregation Principle)

인터페이스 분리 원칙은

클라이언트는 자신이 사용하지 않는 인터페이스에 의존하지 않아야 한다는 원칙이다.

즉, 어떤 클래스가 필요하지 않은 메서드를 억지로 구현하거나 의존하게 만들면 안 된다는 의미다. 이를 지키면 인터페이스를 불필요하게 비대하게 만드는 것을 막고, 클래스는 자신이 필요한 기능에만 집중할 수 있다.

문제가 되는 상황

이 원칙이 무너지는 대표적인 경우는 **모든 기능을 하나의 거대한 인터페이스에 몰아넣으려는 시도**에서 발생한다.

- 통합된 인터페이스는 **불필요한 구현**을 강요한다.
- 인터페이스의 역할이 두루뭉술해지고, 의미가 모호해진다.
- 결과적으로 여러 액터(사용 주체)를 동시에 상대하게 되어 **변경이 어려운 구조**가 된다.

따라서 하나의 범용 인터페이스보다는, **여러 개의 작은 특화된 인터페이스**를 두는 편이 훨씬 바람직하다.

오해하기 쉬운 부분

“비슷한 인터페이스를 하나로 통합하면 더 응집도가 높아지는 것 아닌가?”라는 반문이 있을 수 있다. 겉보기에는 **코드를 한곳에 모으는 것**이 응집을 높이는 행위처럼 보이기 때문이다.

그러나 **응집도(cohesion)**는 단순히 “비슷한 코드를 모아둔 상태”를 의미하지 않는다. 응집도에는 여러 종류가 있으며, 그 수준에 따라 좋은 설계인지 아닌지가 달라진다.

정리하자면,

- **ISP 위배** → 불필요한 의존과 구현 강요 → 인터페이스의 모호성 증가
 - **SRP 위배 가능성** → 하나의 인터페이스가 여러 책임을 동시에 지게 됨
 - **응집도의 착각** → 단순한 통합은 응집이 아니라 오히려 결합도를 높일 수 있음
-

1. 기능적 응집도

- 모듈 내 컴포넌트들이 같은 기능을 수행하도록 설계된 경우
- 모듈이 어떤 목적을 가지고 있고 컴포넌트들은 그 목적을 달성하기 위해 협력하며, 오직 관련된 작업만 수행하는 경우다.
- 기능적 응집도는 데이터보다 역할과 책임 측면에서 바라보는 응집도이다.

2. 순차적 응집도

- 모듈 내의 컴포넌트들이 특정한 작업을 수행하기 위해 순차적으로 연결된 경우를 말한다.
- 어떤 컴포넌트 출력이 다음 컴포넌트의 입력으로 사용되는 형태를 말한다.

3. 통신적 응집도

- 모듈 내의 컴포넌트들이 같은 데이터나 정보를 공유하고 상호작용할 때 이에 따라 모듈을 구성하는 경우를 의미.
- 모듈 내 컴포넌트들이 메시지를 주고받는 형태나 공유 데이터에 따라 구성되면 통신적 응집도를 추구했다고 한다.

4. 절차적 응집도

- 모듈 내의 요소들이 단계별 절차를 따라 동작하도록 설계된 경우
- 모듈의 요소들이 단계별로 연결돼 전체적인 기능을 수행하는 것이다.

예) 입력 단계 → 연산 수행 단계 → 결과 출력 단계

5. 논리적 응집도

- 모듈 내의 요소들이 같은 목적을 달성하기 위해 논리적으로 연관된 경우
- 모듈의 요소들이 서로 관련된 동작을 수행하지만, 특정한 순서나 데이터의 공유가 필요하지는 않는다.

예) 회원 관리 모듈에 등록/정보 업데이트/회원 삭제 등의 작업을 수행하는 것

일반적으로 기능적 > 순차적 > 통신적 > 절차적 > 논리적 순으로 응집도가 높게 평가된다.

하지만 모든 프로젝트가 반드시 이 순서를 따르는 것은 아니다.

“유사한 코드이니 한 곳에 모아두자”라는 접근은 사실상 논리적 응집도에 불과하다.

이는 다른 응집도보다 낮은 수준에 머물게 하며, 바람직한 응집을 달성하지 못한다.

인터페이스 분리 원칙(ISP)이 추구하는 바는 단순한 코드 통합이 아니라, 역할과 책임을 세밀하게 분리하는 것이다.

왜냐하면 인터페이스는 곧 역할(Role)로 볼 수 있기 때문이다.

4.1.4.1 인터페이스와 코드의 재사용성

인터페이스를 이용해 코드의 재사용성을 높인다는 말의 의도는 인터페이스 자체의 재사용성을 높이려 하는 것이 아니다.

오히려 그보다 인터페이스를 사용하는 코드의 재사용성을 높이려는 것이다.

인터페이스를 사용하는 코드는 구현에 의존하지 않으므로 바뀌지 않는 비즈니스 로직을 여러 곳에서 재사용할 수 있다.

필요에 따라 구현체만 바꾸면 되기 때문이다.

4.1.5 의존성 역전 원칙

의존성 역전 원칙을 설명하려면 의존성이란 무엇인가부터 이해해야 한다.

하지만 우선 의존성 역전 원칙에 관해 간단하게 보자.

🔥 의존성 역전 원칙

- 첫째, 상위 모듈은 하위 모듈에 의존해서는 안 된다. 상위모듈과 하위 모듈 모두 추상화에 의존해야 한다.
- 둘째, 추상화는 세부 사항에 의존해서는 안 된다. 세부 사항이 추상화에 의존해야 한다.

즉, 의존성 역전 원칙은 고수준/저수준 모듈이 추상화에 의존해야 한다는 원칙이다.

4.2 의존성

의존은 **다른 객체나 함수를 사용하는 상태**이다.

다시 말해 어떤 객체가 다른 코드를 사용하고 있지만 해도 이를 가리켜 의존하고 있다고 볼 수 있다.

```
class Printer {  
    public void print(Book book) { ... }  
}
```

Printer 클래스는 print 메소드의 매개변수에서 Book 클래스를 사용한다.

따라서 Printer 클래스는 Book 클래스에 의존한다.

```
class Book {  
    private String content;  
    private Writer writer;  
    ...  
}
```

Book 클래스는 String과 Writer 클래스를 사용하여 객체가 어떤 데이터를 가질 수 있는지를 표현한다.

따라서 Book 클래스는 String과 Writer 클래스에 **의존**한다.

예제에서 볼 수 있듯이, 의존의 정의는 단순하다.

사용하기만 해도 의존하는 것이다.

그렇기 때문에 소프트웨어는 결국 **여러 의존 객체들의 집합**이라고 볼 수 있다.

컴퓨터공학에서는 의존을 표현하는 또 다른 용어로 **결합(Coupling)**을 사용한다.

결합은 그 정도에 따라 **강결합(Strong Coupling)**으로 평가되기도 하고, **약결합(Loose Coupling)**으로 평가되기도 한다.

이처럼 결합의 강도를 나타내는 지표를 **결합도(Coupling Degree)**라고 한다.

결합도는 곧 **의존성**과 같은 개념이며, 일반적으로 **결합도가 낮을수록 좋은 설계**라고 평가된다.

의존성 자체는 이해하기 어렵지 않다.

하지만 실제로 **약한 의존 관계를 만들고 유지하는 일은** 쉽지 않다.

이를 위해 사용되는 대표적인 기법 중 하나가 바로 ****의존성 주입(Dependency Injection, DI)****이다.

4.2.1 의존성 주입

의존성을 약화시키는 기법으로 잘 알려진 의존성 주입은 스프링을 배울 때 특히나 자주 언급되는 개념이다.

그래서 스프링을 다루는 개발자라면 아주 친숙한 개념이다.

그러면서 동시에 어렵게 느껴지는 단어이기도 하다.

의존성 주입은 말 그대로 **필요한 의존성을 외부에서 넣어주는(주입)** 것을 의미한다.

예시) 햄버거를 만드는 HamburgerChef 클래스

```
class HamburgerChef {

    public Food make() {
        Bread bread = new WheatBread();
        Meat meat = new Beef();
        Vegetable vegetable = new Lettuce();
        Sauce sauce = new TomatoSauce();

        return Hamburger.builder()
            .bread(bread)
            .meat(meat)
            .vegetable(vegetable)
            .sauce(sauce)
            .build();
    }
}
```

햄버거를 표현하기 위해 Food로 반환하며 10개의 클래스와 추상에 의존하고 있다고 볼 수 있다. 그런데 코드를 다음과 같이 작성한다면 어떨까?

```
class HamburgerChef {

    public Food make(Bread bread,
        Meat meat,
        Vegetable vegetable,
        Sauce sauce) {

        return Hamburger.builder()
            .bread(bread)
            .meat(meat)
            .vegetable(vegetable)
            .sauce(sauce)
            .build();
    }
}
```

메소드가 필요한 협력 객체를 외부에서 전달받아 사용하도록 바뀌었다. 외부에서는 협력 객체를 넣어줄 수 있으니 이러한 객체를 주입할 수 있게 됐다. 이러한 상황을 가리켜 의존성 주입이 사용됐다고 한다.

또한 다른 형태도 존재한다. 예를 들어 생성자를 이용해 의존성을 주입받으면 이를 생성자 주입이라고 한다.

```
class HamburgerChef {

    private final Bread bread;
    private final Meat meat;
    private final Vegetable vegetable;
    private final Sauce sauce;
```

```

    public HamburgerChef(Bread bread, Meat meat, Vegetable vegetable, Sauce
sauce) {
        this.bread = bread;
        this.meat = meat;
        this.vegetable = vegetable;
        this.sauce = sauce;
    }

    public Food make() {

        return Hamburger.builder()
            .bread(bread)
            .meat(meat)
            .vegetable(vegetable)
            .sauce(sauce)
            .build();
    }
}

```

그렇다면 의존성 주입이 왜 의존성을 약화시키는 기법일까?

1. 구체 클래스 의존 제거

의존성 주입을 사용하면 더 이상 `WheatBread`, `Beef` 등과 같은 **구체 클래스**에 직접 의존하지 않는다. 덕분에 의존성의 수가 **10개에서 6개로 줄어들었고**, 이는 곧 절반 가까이 감소한 셈이다.

즉, **의존성 주입은 의존성을 제거하는 것이 아니라 개수를 줄이고, 불필요하게 강한 의존을 방지한다.**

2. 남은 의존성은 괜찮을까?

그렇다면 남아 있는 6개의 의존성은 문제일까?
일반적으로는 그렇지 않다.

- 의존성을 **완전히 제거하는 것은 불가능하다.**
- 객체지향 소프트웨어는 **객체 간의 협력**으로 만들어지기 때문이다.
- 따라서 의존성을 없애려는 시도 자체가 곧 협력의 부정이 된다.

결론적으로, **의존성 주입은 의존성을 없애지 않고 단지 약화시킨다.**

자바를 다뤄본 사람들이라면 코드를 작성하면서 `new` 를 사용하는 것을 최대한 뒤로 미루고 자제하라는 말을 들어보았을 것이다.

왜냐하면 `new` 를 사용하는 것이 사실상 하드코딩이고 강한 의존성을 만드는 대표적인 사례라서 그렇다.

어떤 추상 타입의 변수가 하나 있을 때 그 변수에 `new` 를 통해 파생 클래스를 인스턴스화해서 할당한다면 고정된 객체를 사용하겠다는 의미가 된다.

즉, `new` 를 사용하면 더 이상 다른 객체가 사용될 여지가 사라진다.

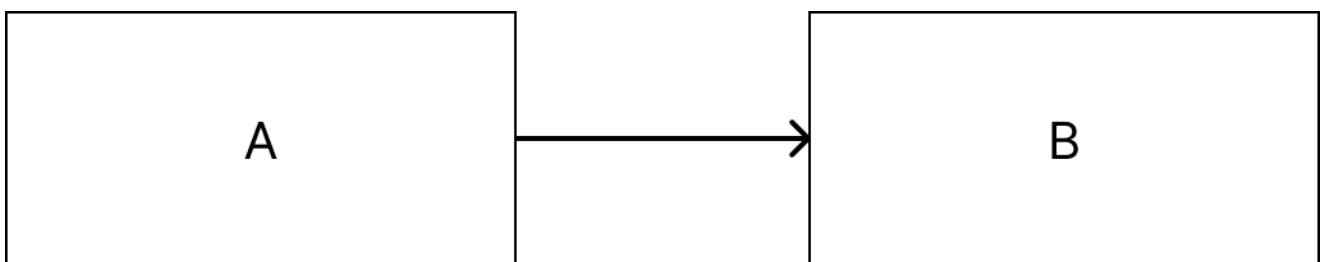
4.2.1 어떤 경우에 `new` 사용이 괜찮을까?

- 순수한 데이터 구조(컬렉션 등)
ArrayList, HashMap 등
비즈니스 의존성이 아닌 로컬 구현 세부사항
- 불변성이나 성능상 이유로 즉시 생성해야 할 때
UUID같은 것들.
상태가 필요없는 즉시 사용 객체
- 라이브러리 기반 도구 클래스
SnowFlakeIdGenerator, SecureRandom,
외부 설정 필요 없고 자체적 독립적이며 재사용 필요 없는 객체

따라서 **new** 를 사용한 코드는 하드 코딩이다. **new**를 사용한 순간 구현에 집중할 수밖에 없게 된다.

4.2.2 의존성 역전

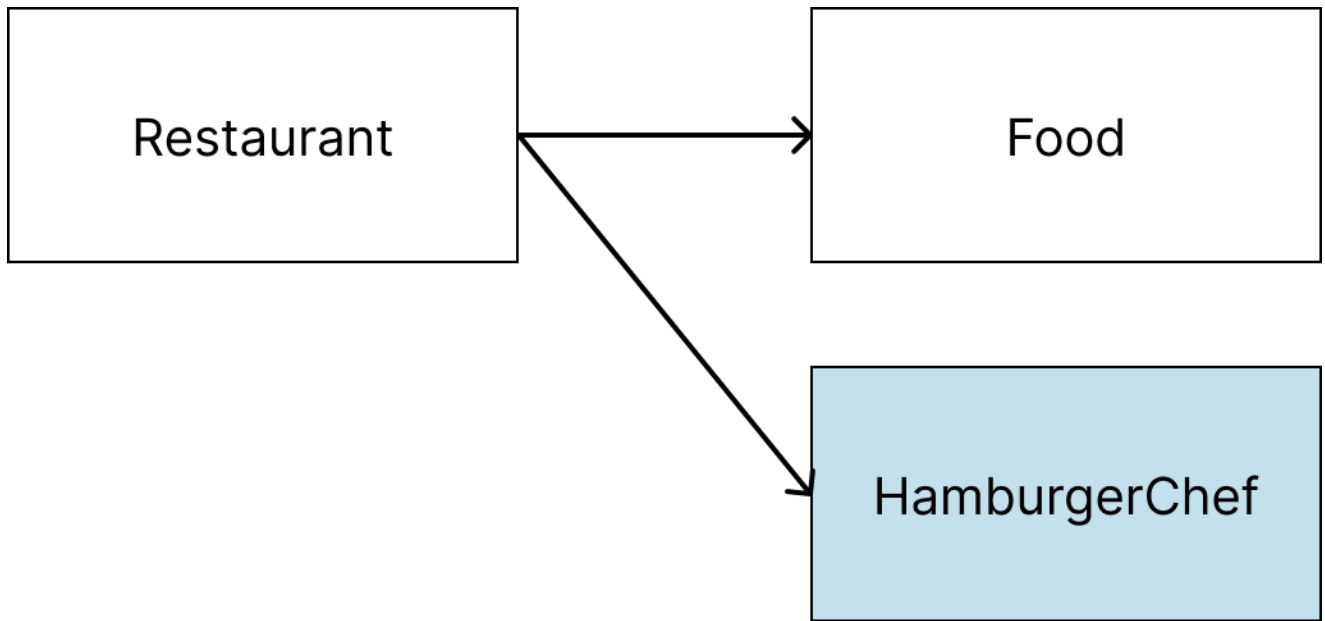
대부분의 소프트웨어 문제는 의존성 역전으로 해결이 가능하다라는 말이 있을 정도로 5가지 원칙 중에서도 가장 중요한 원칙이라고 생각한다.



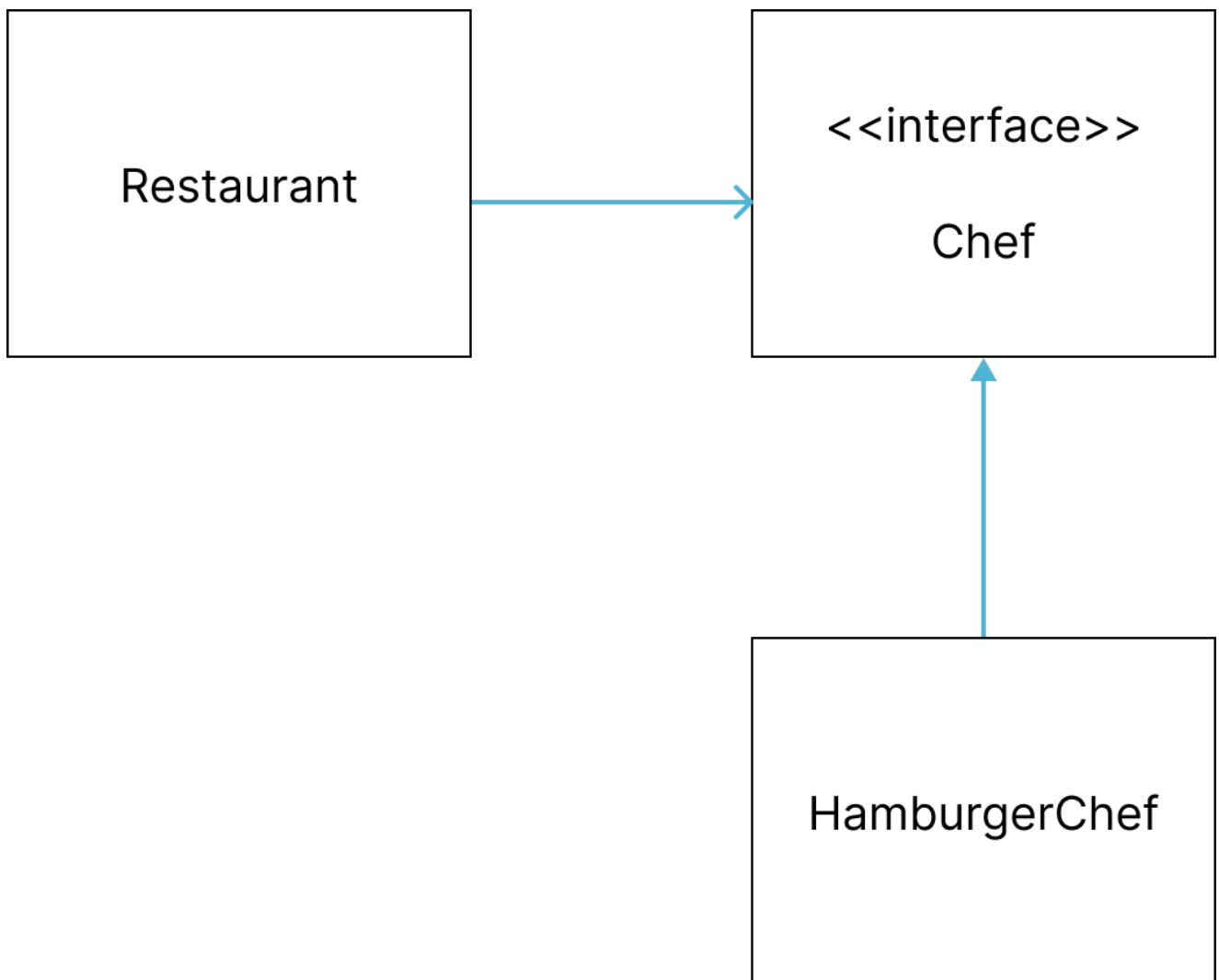
객체 A가 객체 B를 사용한다

A가 B를 사용한다는 의미를 표현하기 위해 **A에서 B로 이어지는 화살표**를 그려 도식화했다.
따라서 **화살표 방향은 의존 방향**이라고 할 수 있다.

예를 들어 식당이라는 클래스가 있고 여기서 HamburgerChef 클래스에 햄버거 만들어달라는 지시를 내린다



이렇게 표현할 수 있다.
하지만, 이 그림을 이렇게 만들면 어떨까?



의존성 역전의 동작 원리

Chef 라는 인터페이스를 만들고 Restaurant 클래스가 Chef 인터페이스에 의존하도록 바꾸었다.
HamburgerChef 클래스는 Chef 인터페이스를 구현하므로 Chef 인터페이스에 의존한다.

즉, 상위 인터페이스를 만들고 기존의 두 클래스가 모두 인터페이스에 의존하도록 바꾼 것이다.

이를 추상화를 이용한 간접 의존 형태로 바꿨다고 표현하며, 이를 의존성을 역전시켰다고 한다.

"의존성 역전"의 의미

의존성을 역전시켰다는 것은 무슨 의미일까? 이를 이해하려면 `HamburgerChef` 클래스에 주목해야 한다.

- **기존:** `HamburgerChef` 에 화살표가 들어오는 방향이었다
- **변경 후:** `Chef` 로 화살표가 나가는 방향으로 바뀌었다

화살표가 들어오는 방향에서 나가는 방향으로 바뀌었다. 앞서 화살표는 의존 방향을 나타낸다고 하였는데, 이와 같은 상황을 보고 **의존의 방향이 역전됐다**고 할 수 있다.

원래는 의존을 당하던 객체가 의존을 하는 객체로 바뀌었다. 이러한 이유로 의존성 역전인 것이다.

정책과 세부사항의 구분

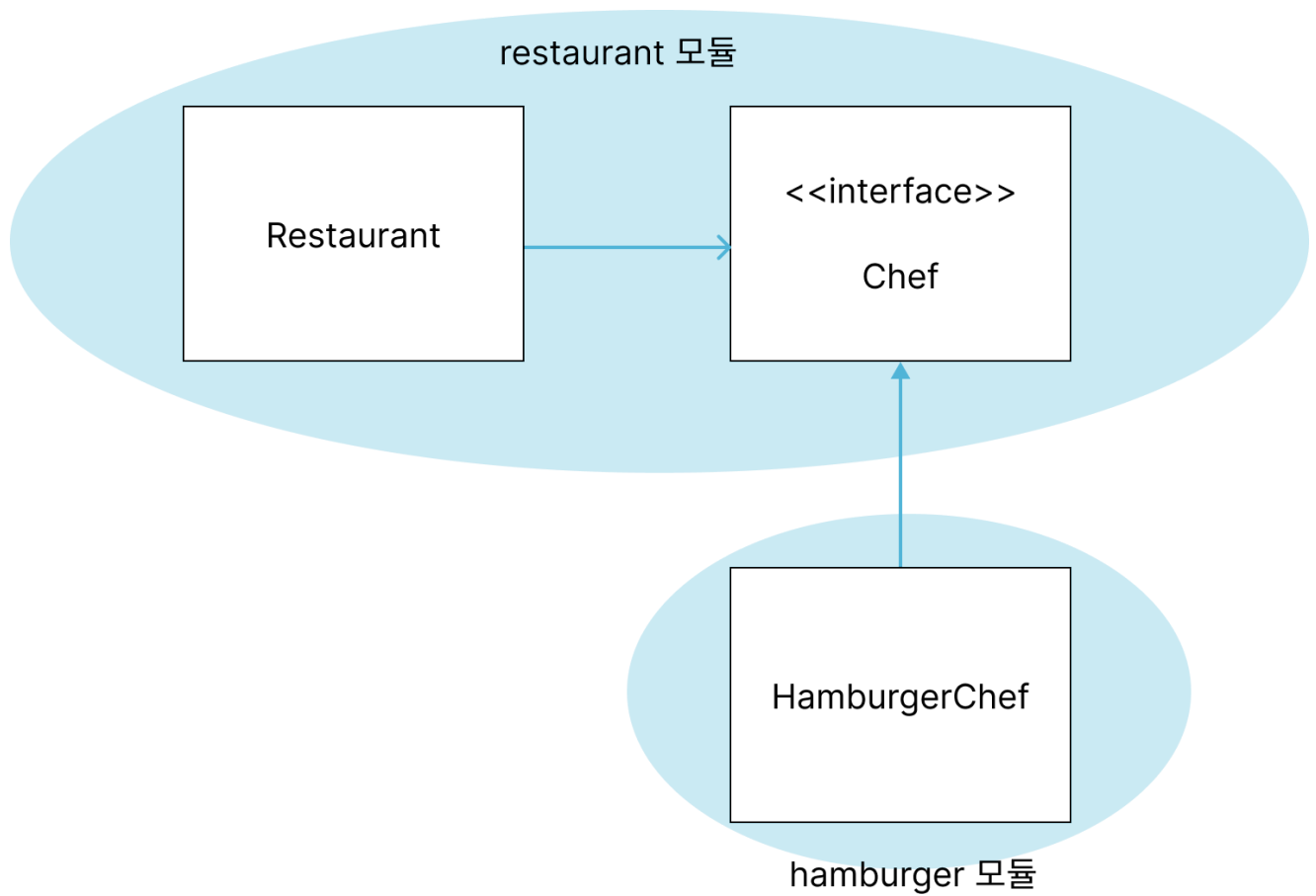
앞서 인터페이스를 계약이라 했으니, 인터페이스를 **정책**이라고 부를 수 있다. 그렇다면 인터페이스를 구현하는 객체는 **세부사항**으로 볼 수 있을 것이다. 인터페이스는 규격이고, 객체는 그 규격에 맞춰 만들어진 구현체이기 때문이다.

의존성 역전을 적용하고 나면 **코드가 추상에 의존하는 형태로** 바뀐다. 그래서 의존성 역전 원칙은 다음과 같이 표현할 수 있다:

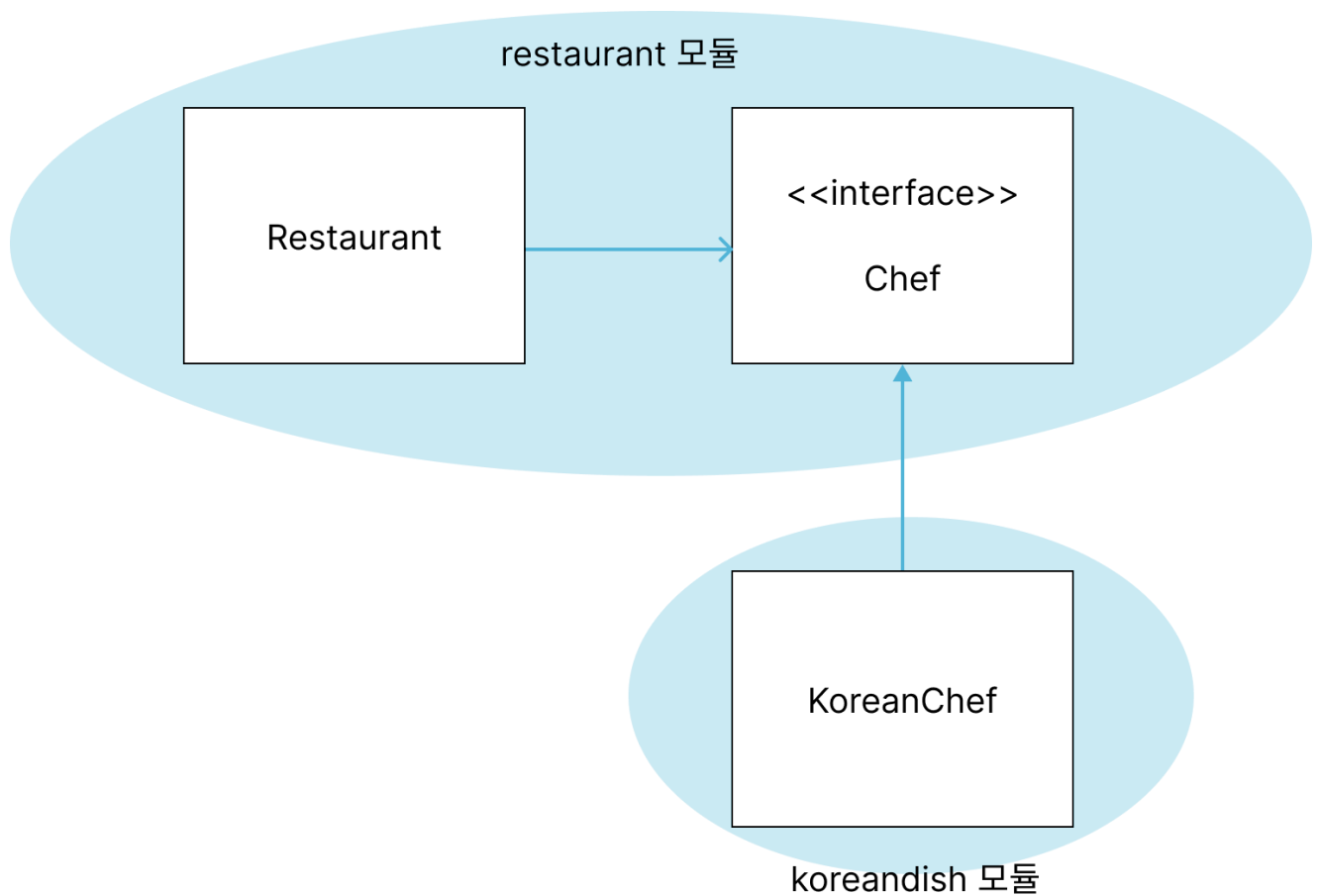
"세부 사항에 의존하지 않고 정책에 의존하도록 코드를 작성하라"

경계를 만드는 기법

또한, **의존성 역전이 경계를 만드는 기법**이라는 말도 이해할 수 있다. 의존성 역전은 경계를 만드는 기법이며, 모듈의 범위를 정하고 상하관계를 표현하는 데 사용할 수 있는 수단이다.



그러면 이 경계로 무엇을 할 수 있을까? 경계를 기준으로 모듈을 나눌 수 있으며 상하관계를 파악할 수 있게 된다.



의존성 역전을 통해 생긴 경계는 곧 **모듈의 경계**로 사용될 수 있다. 의존성 역전이 만든 경계를 기준으로 모듈을 분리하였다.

이 그림에서는 **restaurant 모듈**은 상위모듈, **hamburger 모듈**은 하위 모듈이 된다.

- hamburger 모듈은 restaurant 모듈을 사용하며 restaurant 모듈에 의존한다
- 반대로 restaurant 모듈은 hamburger 모듈을 사용하지 않는다
- restaurant 모듈은 hamburger 모듈에 의존하지 않는다

즉, **하위 모듈은 상위 모듈에 의존하지만 상위 모듈은 하위 모듈에 의존하지 않는 것이다.**

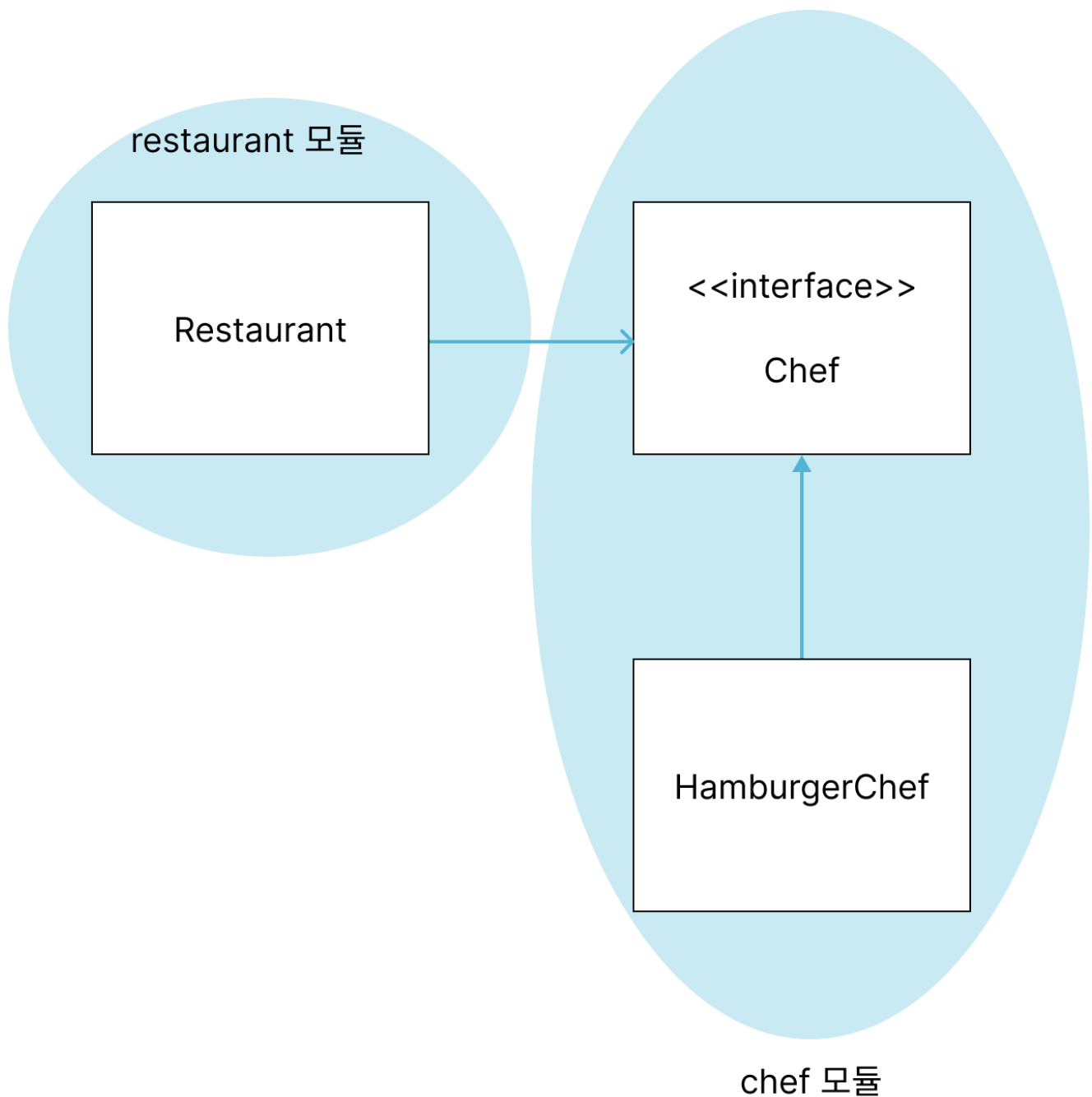
모듈 간 의존 관계를 굳이 한 번 더 짚어서 이야기한 이유는 **모듈 간 상하 관계에 따른 의존방향이 굉장히 중요하다**기 때문이다. 시스템을 설계할 때 **상위모듈은 하위 모듈에 의존해서는 안 된다.**

그리고 의존성 역전은 상위 모듈이 하위 모듈에 의존하지 않게 하고 싶을 때 사용할 수 있는 기법 중 하나이다.

이처럼 의존 방향을 고려해 의존성 역전을 적용하면 **시스템을 플러그인처럼** 어떤 모듈을 꽂느냐에 따라 다르게 동작할 수 있게 만들 수 있다. 위 그림처럼 햄버거를 파는 식당이 될 수도 있지만 한식당이 될 수도 있고, 양식당, 일식당이 될 수 있다.

모듈의 경계를 왜 굳이 나눠야하는 것일까? 다시 말해 Chef 인터페이스가 꼭 restaurant 모듈에 있어야 할까?

restaurant 모듈과 chef 모듈로 나누어 chef 모듈이 Chef 인터페이스를 갖게 하는 것은 어떨까?



- **상위 모듈:** 비즈니스 로직 중심의 클래스, 시스템 행위와 목적 담당
예) OrderService, GameService, ...
- **하위 모듈:** 세부 구현 클래스, 기술적 기능 수행

이것도 괜찮은 방법일 수 있지만, **문제가 하나 생긴다. 상위모듈이 하위 모듈에 의존하게 된다.**

모듈이 서로 의존하는 것은 그렇게까지 이상한 현상은 아니지만 **상위 모듈이 하위 모듈에 의존하는 것은 이상하다.** 왜냐하면 하위 모듈의 변경에 상위 모듈이 영향을 받는다는 의미가 되기 때문이다.

그러면 더는 플러그인처럼 restaurant 모듈의 기능을 자유롭게 변경할 수 없다.

koreandish 모듈로 교체했을 때 restaurant 모듈이 영향을 받지 않고 여전히 항상성을 유지할 수 있을까? chef 모듈이 사라지면 Chef 인터페이스도 함께 사라진다. 그러면 restaurant 모듈에서 컴파일 에러가 쏟아질 것이다.

물론 다행스럽게도 koreanchef 모듈에 Chef 인터페이스가 있을지 모른다. 어떤 식이든 **상위 모듈이 하위 모듈에 영향을 받는 것이다.**

따라서 이런 식으로 모듈 관계를 구성해서는 안 된다. 즉, **상위 모듈은 하위 모듈에 의존해서는 안 된다.** 그래야 변경에 유연하고 재사용 가능한 코드를 만들 수 있다.

🔥 의존성 역전 원칙의 핵심

상위 모듈은 하위 모듈에 의존해서는 안 된다. 상위모듈과 하위 모듈 모두 추상화에 의존해야 한다.

추상화는 세부 사항에 의존해서는 안 된다. 세부사항이 추상화에 의존해야 한다.

4.2.3 의존성 역전과 스프링

🤔 질문

스프링은 의존성 주입을 지원하는 프레임워크인가?

스프링은 의존성 역전 원칙을 지원하는 프레임워크인가?

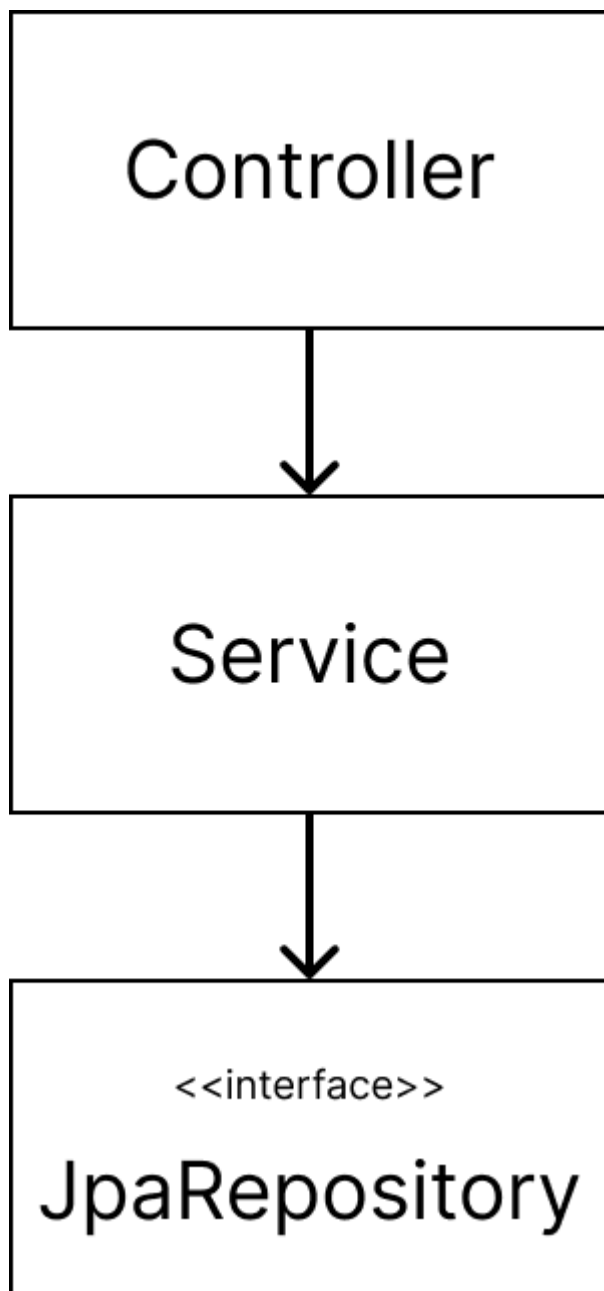
정답부터 말하자면 스프링은 **의존성 주입을 지원하는 프레임워크**이지만 **의존성 역전 원칙을 지원하는 프레임워크가 아니다.**

스프링을 사용한다고 해서 의존성 역전 원칙이 지켜지는 것이 아니다.

의존성 역전 원칙은 **설계의 영역**이다.

따라서, 의존성 역전 원칙을 지키고 싶다면 설계 부문에서 개발자들이 능동적으로 신경써야 한다.

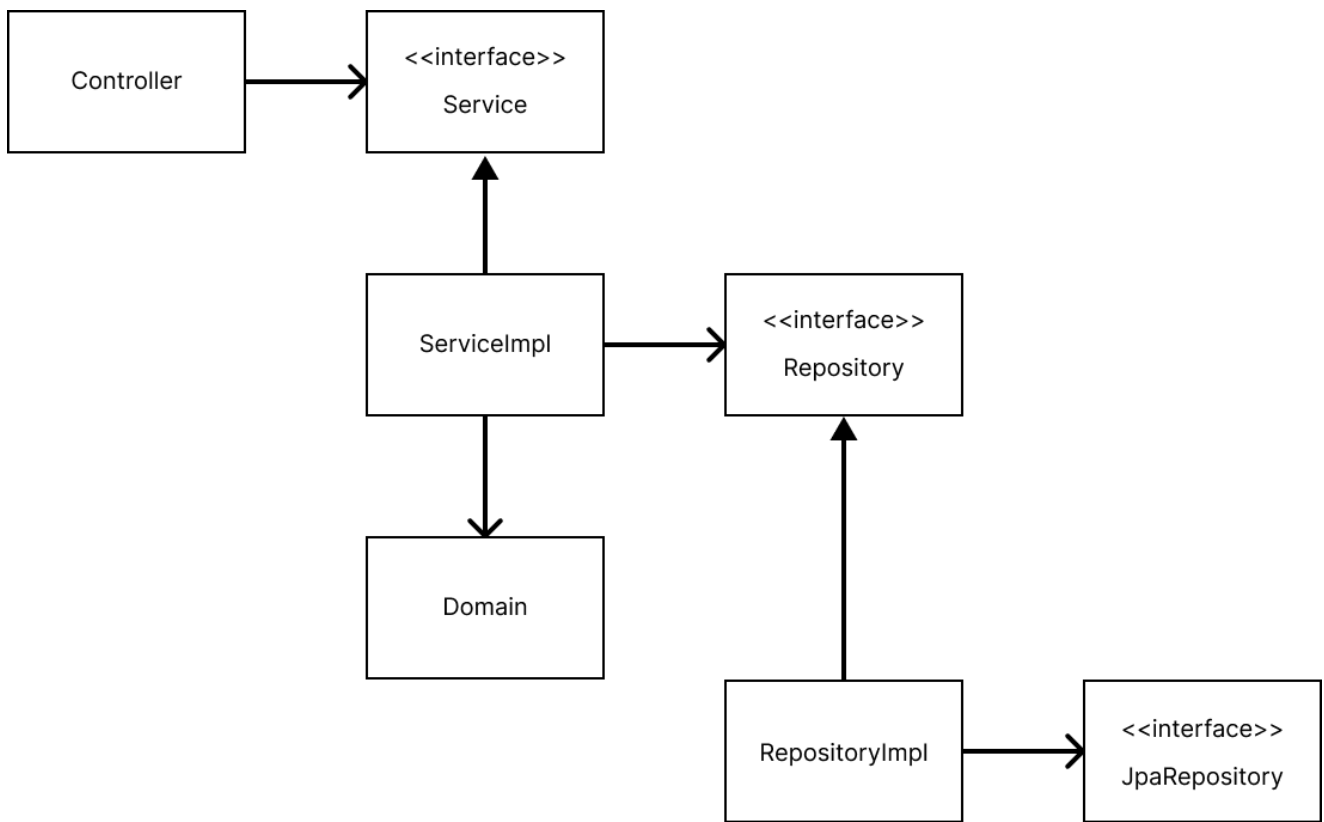
우리들의 프로젝트가 다음 그림처럼 컴포넌트를 직접 호출하고 있지는 않은지 생각해보자.



위 그림에서 의존성 역전은 찾아볼 수 없다.

그러니 **SOLID** 관점에서 위 같은 설계를 좋은 설계라 말하기 어렵다.

그럼에도 안타깝게도 여러 스프링 강의나 책에서 스프링 코드를 위 그림처럼 작성하는 방식으로 알려준다.



이 구조가 어떻게 해서 나오게 됐는지 추후 설명하겠다.

우선 이 구조를 보면서 **의존성 역전이 어디에 어떻게 적용됐는지** 확인하면서 보는 편이 더 좋을 것이다.

4.2.4 의존성이 강조되는 이유

설계관점에서 유지보수성을 판단할 때 크게 세 가지 맥락이 있다.

- 📌 **영향범위**: 코드 변경으로 인한 영향 범위가 어떻게 되는가?
- 🔗 **의존성**: 소프트웨어에서 의존성 관리가 제대로 이뤄지고 있는가?
- 📈 **확장성**: 쉽게 확장이 가능한가?

지금까지 다룬 내용을 보면 이 세 가지 맥락에 문제가 있을 때 어떻게 해결할 수 있을지 알 수 있다.

- 📌 **영향범위**에 문제가 있다면 응집도를 높이고 적절히 모듈화해서 단일 책임 원칙을 준수하는 코드를 만든다
- 🔗 **의존성**에 문제가 있다면 의존성 주입과 의존성 역전 원칙 등을 적용해 약한 의존 관계를 만든다
- 📈 **확장성**에 문제가 있다면 의존성 역전 원칙을 이용해 개방 폐쇄 원칙을 준수하는 코드로 만든다.

소프트웨어 설계의 핵심 원칙

결국 소프트웨어 설계를 잘하고 싶다면 **코드를 변경하거나 확장할 때 영향받는 범위를 최소화할 수 있어야 한다**. 그리고 코드의 영향 범위를 최소화하려면 **의존성을 잘 다뤄야 한다**.

의존성 관리의 목표

의존성 자체를 없애는 것은 불가능하다. 따라서 의존성을 잘 관리한다는 것은 **불필요한 의존성을 줄인다**라는 목표를 포함하지만 의존성을 없애는 것이 목표는 아니다.

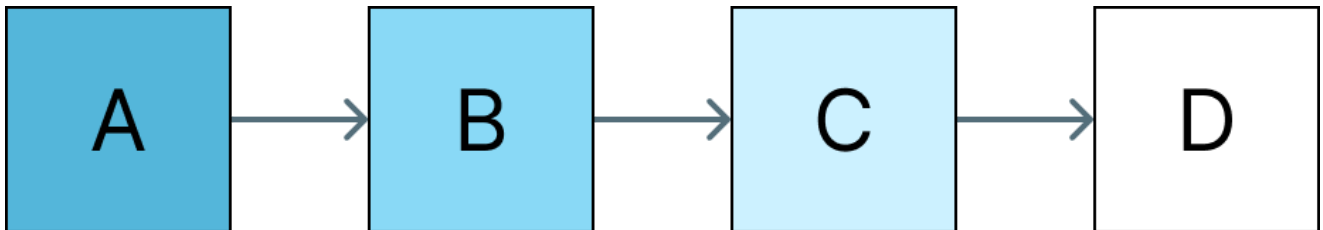
그보다는 의존성을 끊는 것이 목표이다.

의존성 전이의 특징

그러면 의존성을 끊는다는 것은 또 무엇일까? 이를 이해하려면 의존성이 갖고 있는 특징을 하나 이해해야 한다. 바로 **의존성 전이**라는 특징이다.

의존성은 컴포넌트 간의 상호작용을 표현하는 것이므로 한 컴포넌트가 변경되거나 영향을 받으면 관련된 다른 컴포넌트에도 영향이 갈 수 있다.

그런데 이렇게 영향을 받은 컴포넌트가 변경되거나 영향을 받으면 관련된 다른 컴포넌트에도 영향이 갈 수 있다. 그런데 이렇게 영향을 받은 컴포넌트는 **연쇄적으로** 또 다른 관련 컴포넌트에 영향을 준다.



A, B: 영향받는 컴포넌트

C: 변경한 컴포넌트

C 컴포넌트를 변경했을 뿐인데 이 컴포넌트의 코드 변경에 실제로 영향을 받는 컴포넌트는 훨씬 더 많을 수 있다.

왜냐하면 의존성은 전이되기 때문이다.

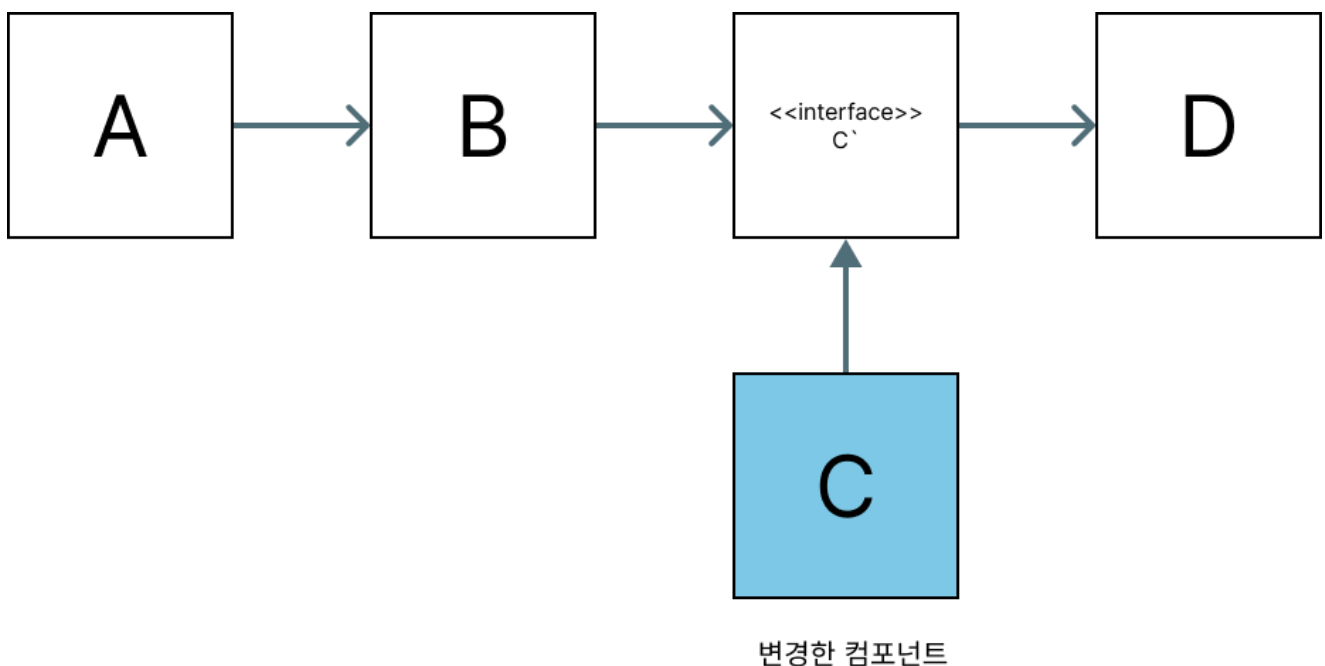
C 컴포넌트를 변경한 결과, B가 영향을 받고, B가 영향을 받은 결과 A도 영향을 받는다.

여기서 알 수 있는 사실이 **의존성은 화살표의 역방향으로 전이된다**라는 것이다.

그리고 이러한 특징 때문에 소프트웨어 설계가 중요해진다.

코드 변경에 자유로워지려면 연결을 최소화하고 어쩔 수 없이 존재하는 연결마저도 약한 연결 상태로 만들어야 한다.

그렇다면 여기서 C 컴포넌트에 의존성 역전을 하게 되면 어떻게 될까?



의존성 역전의 실제 효과

C 컴포넌트에 상위 인터페이스인 C'을 만들고 C가 이를 구현해서 **C가 C'에 의존하도록 바꿨다**. 그랬더니 C 컴포넌트의 코드 변경에도 **아무 컴포넌트도 영향을 받지 않게 되었다**.

이것은 큰 변화이다. **변경으로 인한 영향 범위를 최소화한 것이다**.

의존성 역전의 핵심 가치

따라서 의존성 역전을 적용했다, 추상화를 적용했다라는 말은 **변경으로 인한 영향 범위를 축소했다**라는 말과도 같은 것이다.

그래서 **의존성 역전은 경계를 만드는 기술이다**. 코드가 변경되더라도 경계 밖으로 영향이 가지 않게 하기 때문이다.

추상화의 규칙

이러한 맥락을 이해하면 **자바 인터페이스가 구현을 가져서는 안 되는 이유**도 설명이 된다. 추상에 구현이 들어가서 변경이 잦아서는 안 되기 때문이다.

추상이 자주 변경되면 **전이되는 의존성**으로 인해 영향을 받는 클래스가 너무 많아지게 된다.

의존성 관리의 또 다른 원칙

더불어 소프트웨어 개발과 관련된 격언 중 의존성에 관한 또 다른 격언이 있다. 바로 **순환 참조**를 만들지 말라는 것이다.

 업로드중..

순환 참조의 문제점

이처럼 **순환 참조**는 **의존성 전이 범위를 확장시킨다**. 이는 지금까지 함께 살펴본 **변경으로 인한 영향 범위를 축소하**라는 목표에 반한다. 그래서 순환 참조를 만들지 말라 같은 격언이 있는 것이다.

순환 참조의 다양한 형태

순환 참조는 꼭 양방향 참조에서만 일어나는 일이 아니다. **사이클이 생기는 경우**에도 순환 참조를 만들 수 있다.

사이클이 생기면서 어느 한 컴포넌트가 변경되면 **모든 클래스가 영향을 받게** 되며 사이클이 형성된 컴포넌트들은 **모두 같다고 볼 수 있다**.

순환 참조가 설계에 미치는 해로움

따라서 **순환참조가 설계에서 해롭다**는 말이 여기서 나온 것이다.

- 순환참조는 **복잡한 의존성 그래프**를 유도한다
- **의존성 전이 범위를 넓히는** 주범이다
- 의존 그래프에 **사이클이 생겨서는 안 된다**

순환 참조 해결 방법

순환참조를 만들지 않으려면 **양방향 참조를 끊어내고 단방향으로 만들어야 한다.**

시스템에 존재하는 모든 의존 방향을 **단방향**으로 만들어 문제가 발생했을 때 **원인이 어디인지 추적** 가능할 수 있게 해야 한다.

정리

SOLID 원칙들은 서로 유기적으로 연결되어 있으며, 궁극적으로 **변경에 유연하고 확장 가능한 소프트웨어**를 만들기 위한 설계 지침이다.

각 원칙을 적용할 때는 단순히 규칙을 따르는 것이 아니라, **왜 이 원칙이 필요한지, 어떤 문제를 해결하려는 것인지**를 이해하고 상황에 맞게 적용하는 것이 중요하다.

핵심 메시지

좋은 설계란 **코드 변경으로 인한 영향 범위를 최소화**하고, **의존성을 잘 관리**하며, **쉽게 확장**할 수 있는 설계이다. SOLID 원칙은 이를 달성하기 위한 구체적인 방법론을 제시한다.